Link Prediction on FB15k-237: Validation of Self-Adversarial Negative Sampling

Thalia Delhaise, Lenny Briclet, André Fonseca, and Rafael Gufler

RC2526, Knowledge Graphs

Abstract. We study link prediction on the FB15k-237 benchmark using two knowledge graph embedding models, TransE and RotatE, trained under matched conditions. Our current baseline shows that RotatE outperforms TransE with default Bernoulli negative sampling. We now propose to validate self-adversarial negative sampling as the main extension of the project. The goal is to test whether focusing on harder negatives improves global performance and whether the gains are concentrated on difficult graph structures such as rare relations and low-degree entities.

1 Project Goal

We study link prediction on the FB15k-237 knowledge graph benchmark using two standard knowledge graph embedding models: TransE and RotatE. Our goal is to compare both models under matched training conditions and then study whether better negative sampling improves performance on the harder parts of the graph.

Our current baseline results show that RotatE outperforms TransE under the same setup:

Table 1. Current test-set results with default Bernoulli negative sampling.

Model	MRR	Hits@1	Hits@3	Hits@10
TransE	0.152	0.073	0.174	0.304
RotatE	0.261	0.179	0.286	0.424

These results were obtained with matched settings: embedding dimension $d = 128$, Adam optimizer with learning rate 10^{-3}, batch size 1024, up to 50 epochs, early stopping on validation MRR, CUDA, and seed 42.

2 Proposed Extension: Self-Adversarial Negative Sampling

During training, negative triples are created by corrupting true triples. For example, from a true triple (h, r, t), we can replace the tail entity and create a fake triple (h, r, t').

The issue with standard random or Bernoulli negative sampling is that many negative triples are too easy: they are obviously false, so after a few epochs the model learns little from them. We therefore propose to use **self-adversarial negative sampling**, introduced with RotatE, which gives more importance to negative triples that the current model still finds plausible.

Our hypothesis is that this will not improve all cases uniformly. Instead, we expect the largest gains on harder parts of the graph, especially low-frequency relations and low-degree entities, where random negatives are often less informative.

For each positive triple (h, r, t), we sample a pool of n candidate negative triples. In the tail-corruption case:

$$\{(h, r, t'_1), (h, r, t'_2), \ldots, (h, r, t'_n)\}.$$

Let $s_\theta(h, r, t)$ be a score where higher means more plausible. Each negative receives the weight:

$$p_j = \frac{\exp\big(\alpha s_\theta(h, r, t'_j)\big)}{\sum_{i=1}^{n} \exp\big(\alpha s_\theta(h, r, t'_i)\big)}, \tag{1}$$

where $\alpha > 0$ controls how strongly the method focuses on hard negatives.

The loss is:

$$\mathcal{L} = -\log \sigma\big(s_\theta(h, r, t)\big) - \sum_{j=1}^{n} p_j \log \sigma\big(-s_\theta(h, r, t'_j)\big). \tag{2}$$

The first term increases the score of the true triple, while the second term decreases the scores of the weighted negative triples.

In the main experiment, we plan to use $n = 64$ candidate negatives. If this is too expensive computationally, we will reduce it to $n = 32$ and report this choice explicitly. We will tune $\alpha \in \{0.5, 1.0, 2.0\}$ using the validation set only.

3 Experimental Design

The main experiment is a clean ablation:

RotatE with default Bernoulli negatives vs. **RotatE with self-adversarial negatives**.

Everything else will remain fixed: same dataset, same model architecture, same embedding size, same optimizer, same batch size, same training budget, same seed, and same filtered evaluation protocol.

If time allows, we will also run the same comparison on TransE. However, the minimum required experiment is the RotatE ablation, since the method was introduced for RotatE and RotatE is our stronger baseline.

We will select α using validation MRR and then report final filtered MRR and Hits@10 on the test set.

4 Planned Analysis

In addition to global MRR and Hits@10, we will perform slice analysis to understand where the method helps.

We will report results separately for:

- **relation-frequency slices**: rare vs. frequent relations;
- **entity-degree slices**: low-degree vs. high-degree entities;
- **head vs. tail prediction**: because the baseline already shows a large asymmetry between head and tail queries.

All relation frequencies and entity degrees will be computed from the training set only, to avoid using test-set information during analysis.

This will allow us to check whether self-adversarial negative sampling gives a general improvement or mainly helps on specific difficult parts of the graph.

5 Questions for Feedback

We would like feedback on the following points before implementing the method:

- Is self-adversarial negative sampling an appropriate extension for the scope of this project?
- Is the ablation design sound, given that we keep all settings fixed and only change the negative sampler?
- Is fixing the candidate pool size to $n = 64$ and tuning only $\alpha \in \{0.5, 1.0, 2.0\}$ reasonable?
- Is the planned slice analysis sufficient to support our main claim about where hard negatives help?
- Would you recommend testing another negative sampling strategy instead, or is this one suitable?